Robot Arm Detection & Training Logic

Jessica Orbbec dual-camera lab · **arm_agents.py** · 2026-10-16

This note explains how the current system finds the Universal Robots (UR) arm in Orbbec camera frames, how training labels are produced, and how the YOLO detector is trained. It is written for operators and engineers who need the *why*, not just the CLI commands.

1. Goal and design choice

Unlike the rack OPEN/CLOSED and furnace-slot detectors (which assume objects sit in roughly fixed image regions), the robot arm **moves through the scene**. A fixed template or slot box will fail as soon as the arm changes pose. Therefore the arm pipeline is built as a **learned detector**:

- **Capture** many frames from cam0 and cam1 while the arm is (ideally) in varied poses.
- **Auto-label** a bounding box around the arm in each frame (color prior today; motion as fallback).
- **Train** a small YOLOv8n model so that at run time the network finds the arm wherever it appears, without hand-tuned slots.

Important distinction: the color/motion steps are **label generators for training**. Once weights exist, live "tracking" / recognition is YOLO inference (bounding box + confidence), not HSV masking.

2. End-to-end pipeline

```
ROS publishers (cam0, cam1)
        ■  /orbbec/cam{0,1}/color/image_raw
        ▼
CaptureAgent  →  arm_state/captures/{cam0,cam1}/*.png
        ▼
LabelAgent    →  arm_state/keeps/*.png  +  arm_state/labels/*.txt
                   (+ overlays for visual QA)
        ▼
TrainAgent    →  arm_state/dataset/  →  YOLOv8n train
        ▼
best.pt       →  smoke overlays; ready for live detect
```

Implementation lives in **robotics/arm_agents.py** with state under **robotics/arm_state/**. A convenience wrapper is **arm_agents.cmd** (from PowerShell: `.\arm_agents.cmd …`).

Stage	Input	Output
capture	ROS Image topics (or USB)	PNG frames per camera
label --method color	captures/*.png	YOLO txt + keeps + overlays
label --method motion	captures/*.png (arm must move)	same
train	keeps + labels (≥8 pairs)	runs/arm_detect/weights/best.pt

3. Capture logic

CaptureAgent subscribes to both color topics with BEST_EFFORT QoS (matching the Orbbec publishers) and saves frames on a timer (e.g. every 0.12–0.25 s) for a chosen duration (e.g. 180 s). USB CAP_OBSENSOR is a fallback if ROS is down — USB and ROS must not open the same cameras at once.

- Why both cameras: cam0 sees the furnace / workcell overview; cam1 sees the rack side. The arm is visible (differently) in both, so the detector learns two viewpoints.
- Why many frames: YOLO needs varied examples. Near-duplicate static frames train poorly for new poses — moving the arm during capture is what makes the model generalize.
- Artifacts land in `arm_state/captures/cam0` and `…/cam1`.

4. Labeling logic (default: color)

Motion labeling was tried first (background subtraction + frame differencing). When the arm barely moved, almost every frame looked identical, so motion highlighted monitors, glare, or noise — **not** the arm. The default method is therefore **color**, which uses the UR arm's distinctive light-blue / cyan joint caps and nearby grey metal tubes.

4.1 Why color works here

UR collaborative arms have cyan-blue plastic joint rings and matte grey cylindrical links. In HSV, those joint rings form a compact high-saturation cyan band that is rarer in the rest of the lab than "any blue-ish pixel." Grey tubes alone are ambiguous (racks, tables); grey *next to* cyan joints is a strong arm prior.

4.2 Algorithm (_arm_color_box)

For each BGR frame:

- **HSV convert** the image.
- **Tight cyan mask**: H∈[90,112], S≥60, V≥70 (OpenCV hue scale 0–179). Morphological open/close cleans speckles. If too few pixels, a slightly looser fallback range is tried.
- **Grey mask**: low saturation metal-like pixels (S≤50, mid value).
- **Contours on cyan** → list of blobs (x, y, w, h, area), drop tiny ones, sort by area.
- **Cluster**: for each of the top seeds, absorb other blobs within ~200 px. This groups multiple joint rings that belong to one arm.
- **Expand with grey**: dilate the joint cluster region and union overlapping grey blobs so the box covers links, not only joint caps.
- **Score** each candidate: cyan area + 0.35× nearby grey; heavy penalties if the box is huge (>55% of frame), if grey is almost absent (blue plastic bins), or if the box only hugs the top edge (ceiling clutter).
- **Camera prior**: on cam1 the arm usually sits on the **right**, so score scales up with horizontal center-x; cam0 prefers center-right.
- Keep the highest-scoring box if score ≥ 40 and size constraints pass; else skip.

The winning box is converted to YOLO format `class cx cy w h` (normalized 0–1), copied into `keeps/`, and drawn into `overlays/` for human QA (yellow "arm" rectangle).

4.3 Motion labeling (optional fallback)

With `--method motion`, LabelAgent builds a median grey background from sampled frames, then for each frame takes the union of large blobs from (a) background difference and (b) previous-frame difference. This only works when the arm **actually moves** relative to a clean background. If the arm is static or baked into the "background," labels go to the wrong objects.

5. Training logic

5.1 Dataset assembly

TrainAgent pairs every `keeps/*.png` with a matching non-empty `labels/*.txt`. It requires at least 8 pairs. Pairs are shuffled (seed 42) and split ~85% train / 15% val into `arm_state/dataset/` with an Ultralytics YAML (single class name: **arm**).

5.2 Model and optimization

- Base model: **YOLOv8n** (nano) — small enough for CPU, fast enough for overlay use.
- Typical knobs: imgsz=640, batch=8, workers=0 (Windows-friendly), patience=15.
- On Windows, `KMP_DUPLICATE_LIB_OK=TRUE` avoids an OpenMP double-load abort (torch + MKL).

- Outputs: `arm_state/runs/arm_detect/weights/best.pt` and `last.pt`; path recorded in `arm.json`.
- After train, a few keeps are run through the detector; plotted results go to `overlays/train_smoke/` for a quick sanity check.

5.3 What the network learns

YOLO learns appearance cues from the auto-boxes: grey links, cyan joints, gripper silhouette, and typical lab context. At inference it predicts one (or more) axis-aligned boxes with a confidence score (config default conf=0.25). It does **not** currently estimate joint angles or a 3D pose — only “where is the arm in the image.”

6. “Tracking” vs detection today

In everyday language people say “track the arm.” The implemented system is **frame-by-frame object detection**, not a temporal tracker (no Kalman filter, no ByteTrack ID persistence yet). Each new image is scored independently by YOLO. Temporal smoothness can be added later by feeding detections into a tracker, but that is not required for training or for a first overlay.

Color masking is **not** the live tracker. It is a bootstrap labeler so humans do not have to draw thousands of boxes by hand. After training, prefer YOLO — it is more robust to lighting and clutter than a fixed HSV gate.

7. Current trained state (snapshot)

- Weights: `arm_state/runs/arm_detect/weights/best.pt`
- Last label method: **color** (thousands of cam0/cam1 frames labeled).
- Last train used on the order of a few hundred subsampled keeps (near-duplicates thinned so CPU training finishes in a reasonable time).
- Validation mAP50 after fine-tune was high (~0.97) on held-out frames from the same distribution — expect lower accuracy on **new poses** until more moving-arm captures are added and the model is retrained.
- Smoke tests showed correct arm boxes at moderate confidence (~0.7) on cam0 frames.

8. Operator recipe (PowerShell)

Publishers must be running (ROS dual sync, or publishers-only). Then:

```
cd C:\Users\Ceder-ALAB\Desktop\Jessica\Jessica\robotics
.\arm_agents.cmd capture --seconds 180 --source ros
.\arm_agents.cmd label --method color
.\arm_agents.cmd train --epochs 15
.\arm_agents.cmd status
```

During **capture**, move the arm through many poses (reach left/right, up/down, toward rack and furnace). That is the main lever for better generalization. Labeling and training can then be left unsupervised.

9. Limitations and next improvements

- Auto-labels can still miss the gripper tip or include a bit of background; spot-check `overlays/` before long training runs.
- Static capture → high val metrics but weak on unseen poses.
- No multi-object / people / second arm disambiguation yet.
- Live overlay in dual_view is not necessarily wired yet — detection is trained and smoke-tested; integration into the ROS dual window is a separate step.
- Future: temporal tracker on top of YOLO; optional human-in-the-loop box cleanup; more aggressive pose-diverse capture scripts.

10. File map

Path	Role
robotics/arm_agents.py	Capture / label / train agents + CLI
robotics/arm_agents.cmd	Pixi/ROS-env wrapper for PowerShell
robotics/arm_state/captures/	Raw frames cam0 & cam1
robotics/arm_state/keeps/	Labeled training images

robotics/arm_state/labels/	YOLO .txt boxes
robotics/arm_state/overlays/	QA drawings + train_smoke
robotics/arm_state/dataset/	Ultralytics train/val split
robotics/arm_state/runs/arm_detect/	Training logs + best.pt
robotics/arm_state/arm.json	Config, last counts, weights path

Bottom line: **color auto-label** → **YOLO train** → **YOLO detect**. Color is how we teach; YOLO is how we recognize the moving arm.